A Guide to the Go Garbage Collector

Table of Contents

- Introduction
  - Where Go Values Live
  - Tracing Garbage Collection
- The GC cycle
  - Understanding costs
  - GOGC
  - Memory limit
  - Latency
  - Finalizers, cleanups, and weak pointers
  - Additional resources
- A note about virtual memory
- Optimization guide
  - Identifying costs
  - Eliminating heap allocations
  - Implementation-specific optimizations
  - Linux transparent huge pages (THP)
- Appendix
  - Additional notes on GOGC

Introduction

This guide is intended to aid advanced Go users in better understanding their application costs by providing insights into the Go garbage collector. It also provides guidance on how Go users may use these insights to improve their applications' resource utilization. It does not assume any knowledge of garbage collection, but does assume familiarity with the Go programming language.

The Go language takes responsibility for arranging the storage of Go values; in most cases, a Go developer need not care about where these values are stored, or why, if at all. In practice, however, these values often need to be stored in computer **physical memory** and physical memory is a finite resource. Because it is finite, memory must be managed carefully and recycled in order to avoid running out of it while executing a Go program. It's the job of a Go implementation to allocate and recycle memory as needed.

Another term for automatically recycling memory is **garbage collection**. At a high level, a garbage *collector* (or GC, for short) is a system that recycles memory on behalf of the application by identifying which parts of memory are no longer needed. The Go standard toolchain provides a runtime library that ships with every application, and this runtime library includes a garbage collector.

Note that the existence of a garbage collector as described by this guide is not guaranteed by the Go specification, only that the underlying storage for Go values is managed by the language itself. This omission is intentional and enables the use of radically different memory management techniques.

Therefore, this guide is about a specific implementation of the Go programming language and *may not apply to other implementations*. Specifically, this following guide applies to the standard toolchain (the `gc` Go compiler and tools). Gccgo and Gollvm both use a very similar GC implementation so many of the same concepts apply, but details may vary.

Furthermore, this is a living document and will change over time to best reflect the latest release of Go. This document currently describes the garbage collector as of Go 1.19.

Where Go Values Live

Before we dive into the GC, let's first discuss the memory that doesn't need to be managed by the GC.

For instance, non-pointer Go values stored in local variables will likely not be managed by the Go GC at all, and Go will instead arrange for memory to be allocated that's tied to the lexical scope in which it's created. In general, this is more efficient than relying on the GC, because the Go compiler is able to predetermine when that memory may be freed and emit machine instructions that clean up. Typically, we refer to allocating memory for Go values this way as "stack allocation," because the space is stored on the goroutine stack.

Go values whose memory cannot be allocated this way, because the Go compiler cannot determine its lifetime, are said to *escape to the heap*. "The heap" can be thought of as a catch-all for memory allocation, for when Go values need to be placed *somewhere*. The act of allocating memory on the heap is typically referred to as "dynamic memory allocation" because both the compiler and the runtime can make very few assumptions as to how this memory is used and when it can be cleaned up. That's where a GC comes in: it's a system that specifically identifies and cleans up dynamic memory allocations.

There are many reasons why a Go value might need to escape to the heap. One reason could be that its size is dynamically determined. Consider for instance the backing array of a slice whose initial size is determined by a variable, rather than a constant. Note that escaping to the heap must also be transitive: if a reference to a Go value is written into another Go value that has already been determined to escape, that value must also escape.

Whether a Go value escapes or not is a function of the context in which it is used and the Go compiler's escape analysis algorithm. It would be fragile and difficult to try to enumerate precisely when values escape: the algorithm itself is fairly sophisticated and changes between Go releases. For more details on how to identify which values escape and which do not, see the section on eliminating heap allocations.

Tracing Garbage Collection

Garbage collection may refer to many different methods of automatically recycling memory; for example, reference counting. In the context of this document, garbage collection refers to **tracing** garbage collection, which identifies in-use, so-called **live**, objects by following pointers transitively.

Let's define these terms more rigorously.

- **Object**—An object is a dynamically allocated piece of memory that contains one or more Go values.
- **Pointer**—A memory address that references any value within an object. This naturally includes Go values of the form ∗T, but also includes parts of built-in Go values. Strings, slices, channels, maps, and interface values all contain memory addresses that the GC must trace.

Together, objects and pointers to other objects form the **object graph**. To identify live memory, the GC walks the object graph starting at the program's **roots**, pointers that identify objects that are definitely in-use by the program. Two examples of roots are local variables and global variables. The process of walking the object graph is referred to as **scanning**. Another phrase you might see in the Go documentation is whether an object is **reachable**, which just means that the object can be discovered by the scanning process. Note also that, with one exception, once memory becomes unreachable, it stays unreachable.

This basic algorithm is common to all tracing GCs. Where tracing GCs differ is what they do once they discover memory is live. Go's GC uses the mark-sweep technique, which means that in order to keep track of its progress, the GC also **marks** the values it encounters as live. Once tracing is complete, the GC then walks over all memory in the heap and makes all memory that is *not* marked available for allocation. This process is called **sweeping**.

One alternative technique you may be familiar with is to actually *move* the objects to a new part of memory and leave behind a forwarding pointer that is later used to update all the application's pointers. We call a GC that moves objects in this way a **moving** GC; Go has a **non-moving** GC.

The GC cycle

Because the Go GC is a mark-sweep GC, it broadly operates in two phases: the mark phase, and the sweep phase. While this statement might seem tautological, it contains an important insight: it's not possible to release memory back to be allocated until *all* memory has been traced, because there may still be an un-scanned pointer keeping an object alive. As a result, the act of sweeping must be entirely separated from the act of marking. Furthermore, the GC may also not be active at all, when there's no GC-related work to do. The GC continuously rotates through these three phases of sweeping, off, and marking in what's known as the **GC cycle**. For the purposes of this document, consider the GC cycle starting with sweeping, turning off, then marking.

The next few sections will focus on building intuition for the costs of the GC to aid users in tweaking GC parameters for their own benefit.

Understanding costs

The GC is inherently a complex piece of software built on even more complex systems. It's easy to become mired in detail when trying to understand the GC and tweak its behavior. This section is intended to provide a framework for reasoning about the cost of the Go GC and its tuning parameters.

To begin with, consider this model of GC cost based on three simple axioms.

1. The GC involves only two resources: physical memory, and CPU time.

2. The GC's memory costs consist of live heap memory, new heap memory allocated before the mark phase, and space for metadata that, even if proportional to the previous costs, are small in comparison.

   GC memory cost for cycle N = live heap from cycle N-1 + new heap

   Live heap memory is memory that was determined to be live by the previous GC cycle, while new heap memory is any memory allocated in the current cycle, which may or may not be live by the end. How much memory is live at any given point in time is a property of the program, and not something the GC can directly control.

3. The GC's CPU costs are modeled as a fixed cost per cycle, and a marginal cost that scales proportionally with the size of the live heap.

   *GC CPU time for cycle N = Fixed CPU time cost per cycle + average CPU time cost per byte * live heap memory found in cycle N*

   The fixed CPU time cost per cycle includes things that happen a constant number of times each cycle, like initializing data structures for the next GC cycle. This cost is typically small, and is included just for completeness.

   Most of the CPU cost of the GC is marking and scanning, which is captured by the marginal cost. The average cost of marking and scanning depends on the GC implementation, but also on the behavior of the program. For example, more pointers means more GC work, because at minimum the GC needs to visit all the pointers in the program. Structures like linked lists and trees are also more difficult for the GC to walk in parallel, increasing the average cost per byte.

   This model ignores sweeping costs, which are proportional to total heap memory, including memory that is dead (it must be made available for allocation). For Go's current GC implementation, sweeping is so much faster than marking and scanning that the cost is negligible in comparison.

This model is simple but effective: it accurately categorizes the dominant costs of the GC. It also tells us that the *total CPU cost* of the garbage collector depends on the total number of GC cycles in a given time frame. Finally, embedded in this model is a fundamental time/space trade-off for the GC.

To see why, let's explore a constrained but useful scenario: the **steady state**. The steady state of an application, from the GC's perspective, is defined by the following properties:

- The rate at which the application allocates new memory (in bytes per second) is constant.

  This means that, from the GC's perspective, the application's workload looks approximately the same over time. For example, for a web service, this would be a constant request rate with, on average, the same kinds of requests being made, with the average lifetime of each request staying roughly constant.

- The marginal costs of the GC are constant.

  This means that statistics of the object graph, such as the distribution of object sizes, the number of pointers, and the average depth of data structures, remain the same from cycle to cycle.

Let's work through an example. Assume some application is operating in a steady-state, allocating 10 MiB/s, while the GC can scan memory at a rate of 100 MiB/cpu-second (this is made up). The steady state makes no assumptions about the size of the live heap, but for simplicity, let's say this application's live heap is always 10 MiB. Let's also assume, again, for simplicity, that the fixed GC costs are zero. Let's play around with the GC cycle period.

Suppose each GC cycle happens after exactly 1 cpu-second. Then, by the end of each GC cycle our example application will have allocated 10 MiB of additional memory, resulting in a 20 MiB total heap size. And with every GC cycle, the GC will spend 0.1 cpu-seconds scanning the 10 MiB live heap, resulting in a 10% CPU overhead. Recall that the GC only needs to walk the live heap, not the whole heap. (Note: a constant live heap does not mean that all newly allocated memory is dead. It means that, after the GC runs, *some mix* of old and new heap memory dies, and only that the end result is 10 MiB found live each cycle.)

Now suppose each GC cycle happens less often, once every 2 cpu-seconds. Then, our example application, in the steady state, will have a 30 MiB total heap size on each GC cycle, since it'll allocate 20 MiB in that time. But with every GC cycle, the GC will *still only need 0.1 cpu-seconds* to scan the 10 MiB of live memory. Again, we're assuming the live heap size stays the same, regardless of how much memory is allocated. So this means that our GC overhead just went down, from 10% to 5%, at the cost of 50% more memory being used.

This change in overheads is the fundamental time/space trade-off mentioned earlier. And **GC frequency** is at the center of this trade-off: if we execute the GC more frequently, then we use less memory, and vice versa. But how often does the GC actually execute? In Go, deciding when the GC should start is the main parameter which the user has control over.

GOGC

At a high level, GOGC determines the trade-off between GC CPU and memory.

It works by determining the target heap size after each GC cycle, a target value for the total heap size in the next cycle. The GC's goal is to finish a collection cycle before the total heap size exceeds the target heap size. Total heap size is defined as the live heap size at the end of the previous cycle, plus any new heap memory allocated by the application since the previous cycle. Meanwhile, target heap memory is defined as:

*Target heap memory = Live heap + (Live heap + GC roots) * GOGC / 100*

As an example, consider a Go program with a live heap size of 8 MiB, 1 MiB of goroutine stacks, and 1 MiB of pointers in global variables. Then, with a GOGC value of 100, the amount of new memory that will be allocated before the next GC runs will be 10 MiB, or 100% of the 10 MiB of work, for a total heap footprint of 18 MiB. With a GOGC value of 50, then it'll be 50%, or 5 MiB. With a GOGC value of 200, it'll be 200%, or 20 MiB.

> Note: GOGC includes the root set only as of Go 1.18. Previously, it would only count the live heap. Often, the amount of memory in goroutine stacks is quite small and the live heap size dominates all other sources of GC work, but in cases where programs had hundreds of thousands of goroutines, the GC was making poor judgements.

The heap target controls GC frequency: the bigger the target, the longer the GC can wait to start another mark phase and vice versa. While the precise formula is useful for making estimates, it's best to think of GOGC in terms of its fundamental purpose: a parameter that picks a point in the GC CPU and memory trade-off. The key takeaway is that **doubling GOGC will double heap memory overheads and roughly halve GC CPU cost**, and vice versa. (To see a full explanation as to why, see the appendix.)

> Note: the target heap size is just a target, and there are several reasons why the GC cycle might not finish right at that target. For one, a large enough heap allocation can simply exceed the target. However, other reasons appear in GC implementations that go beyond the GC model this guide has been using thus far. For some more detail, see the latency section, but the complete details may be found in the additional resources.

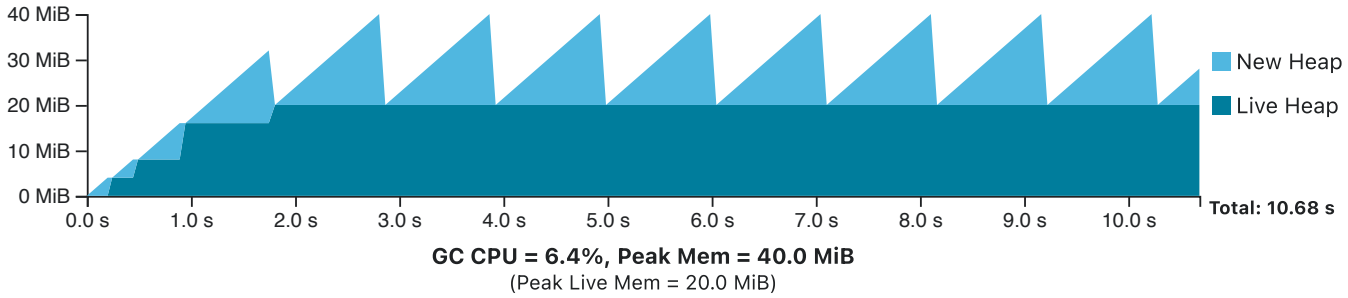
GOGC may be configured through either the `GOGC` environment variable (which all Go programs recognize), or through the `SetGCPercent` API in the `runtime/debug` package.

Note that GOGC may also be used to turn off the GC entirely (provided the memory limit does not apply) by setting `GOGC=off` or calling `SetGCPercent(-1)`. Conceptually, this setting is equivalent to setting GOGC to a value of infinity, as the amount of new memory before a GC is triggered is unbounded.

To better understand everything we've discussed so far, try out the interactive visualization below that is built on the GC cost model discussed earlier. This visualization depicts the execution of some program whose non-GC work takes 10 seconds of CPU time to complete. In the first second it performs some initialization step (growing its live heap) before settling into a steady state. The application allocates 200 MiB in total, with 20 MiB live at a time. It assumes

that the only relevant GC work to complete comes from the live heap, and that (unrealistically) the application uses no additional memory.

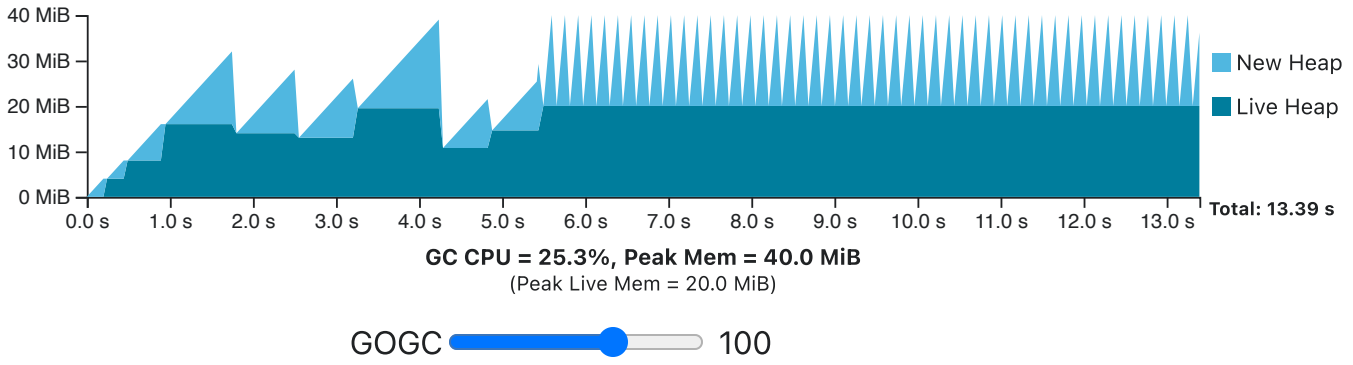
Use the slider to adjust the value of GOGC to see how the application responds in terms of total duration and GC overhead. Each GC cycle ends while the new heap drops to zero. The time taken while the new heap drops to zero is the combined time for the mark phase for cycle N, and the sweep phase for the cycle N+1. Note that this visualization (and all the visualizations in this guide) assume the application is paused while the GC executes, so GC CPU costs are fully represented by the time it takes for new heap memory to drop to zero. This is only to make visualization simpler; the same intuition still applies. The X axis shifts to always show the full CPU-time duration of the program. Notice that additional CPU time used by the GC increases the overall duration.



Notice that the GC always incurs some CPU and peak memory overhead. As GOGC increases, CPU overhead decreases, but peak memory increases proportionally to the live heap size. As GOGC decreases, the peak memory requirement decreases at the expense of additional CPU overhead.

> Note: the graph displays CPU time, not wall-clock time to complete the program. If the program runs on 1 CPU and fully utilizes its resources, then these are equivalent. A real-world program likely runs on a multi-core system and does not 100% utilize the CPUs at all times. In these cases the wall-time impact of the GC will be lower.
>
> Note: the Go GC has a minimum total heap size of 4 MiB, so if the GOGC-set target is ever below that, it gets rounded up. The visualization reflects this detail.

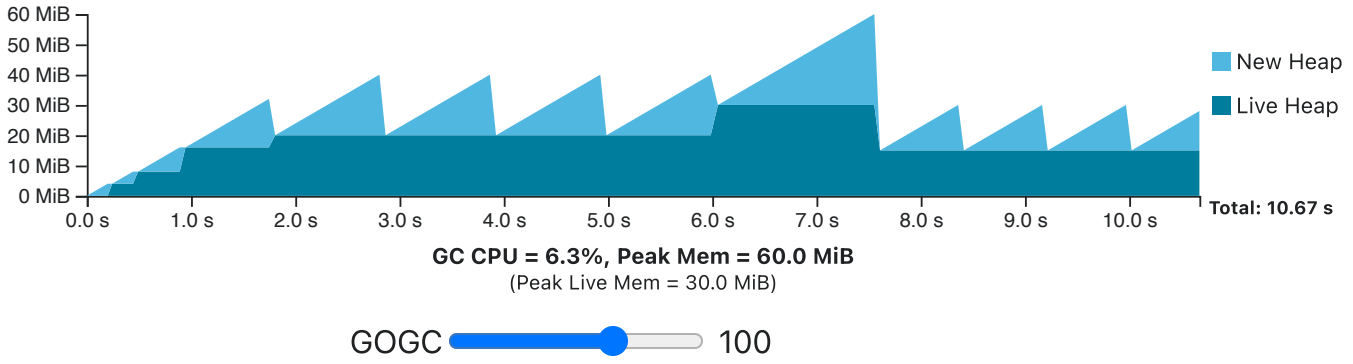
Here's another example that's a little bit more dynamic and realistic. Once again, the application takes 10 CPU-seconds to complete without the GC, but the steady state allocation rate increases dramatically half-way through, and the live heap size shifts around a bit in the first phase. This example demonstrates how the steady state might look when the live heap size is actually changing, and how a higher allocation rate leads to more frequent GC cycles.



Memory limit

Until Go 1.19, GOGC was the sole parameter that could be used to modify the GC's behavior. While it works great as a way to set a trade-off, it doesn't take into account that available memory is finite. Consider what happens when there's a transient spike in the live heap size: because the GC will pick a total heap size proportional to that live heap size, GOGC must be configured such for the *peak* live heap size, even if in the usual case a higher GOGC value provides a better trade-off.

The visualization below demonstrates this transient heap spike situation.



If the example workload is running in a container with a bit over 60 MiB of memory available, then GOGC can't be increased beyond 100, even though the rest of the GC cycles have the available memory to make use of that extra memory. Furthermore, in some applications, these transient peaks can be rare and hard to predict, leading to occasional, unavoidable, and potentially costly out-of-memory conditions.

That's why in the 1.19 release, Go added support for setting a runtime memory limit. The memory limit may be configured either via the `GOMEMLIMIT` environment variable which all Go programs recognize, or through the `SetMemoryLimit` function available in the `runtime/debug` package.

This memory limit sets a maximum on the *total amount of memory that the Go runtime can use*. The specific set of memory included is defined in terms of `runtime.MemStats` as the expression

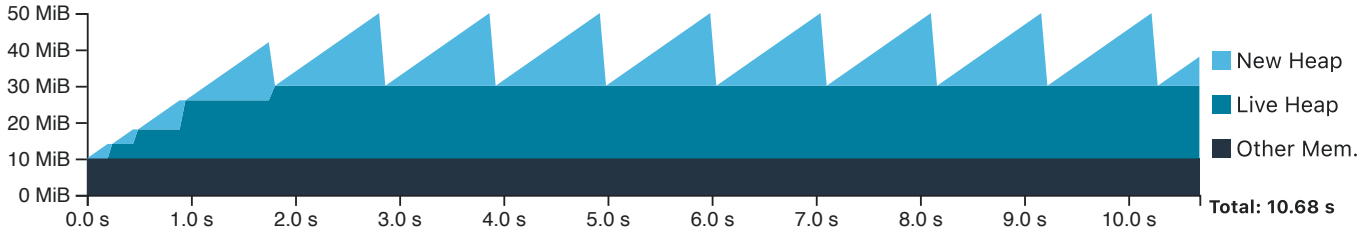
`Sys – HeapReleased`

or equivalently in terms of the `runtime/metrics` package,

`/memory/classes/total:bytes – /memory/classes/heap/released:bytes`

Because the Go GC has explicit control over how much heap memory it uses, it sets the total heap size based on this memory limit and how much other memory the Go runtime uses.

The visualization below depicts the same single-phase steady state workload from the GOGC section, but this time with an extra 10 MiB of overhead from the Go runtime and with an adjustable memory limit. Try shifting around both GOGC and the memory limit and see what happens.



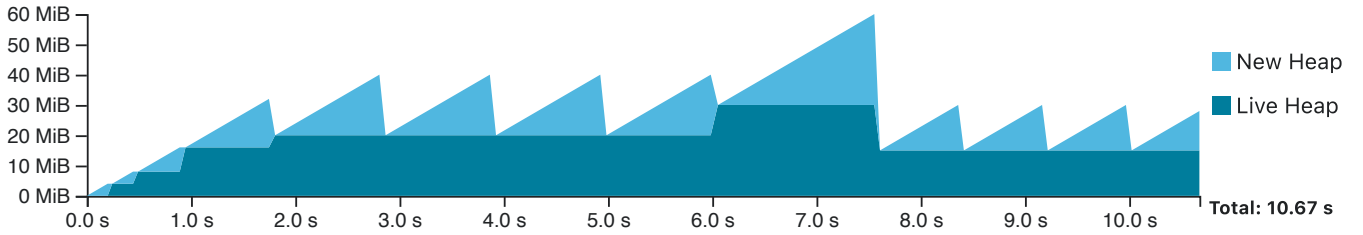
GC CPU = 6.4%, Peak Mem = 50.0 MiB
(Peak Live Mem = 20.0 MiB, Other Mem = 10.0 MiB)

GOGC 100

Memory Limit 100.0 MiB

Notice that when the memory limit is lowered below the peak memory that's determined by GOGC (42 MiB for a GOGC of 100), the GC runs more frequently to keep the peak memory within the limit.

Returning to our previous example of the transient heap spike, by setting a memory limit and turning up GOGC, we can get the best of both worlds: no memory limit breach, and better resource economy. Try out the interactive visualization below.



GC CPU = 6.3%, Peak Mem = 60.0 MiB
(Peak Live Mem = 30.0 MiB)

GOGC 100

Memory Limit 100.0 MiB

Notice that with some values of GOGC and the memory limit, peak memory use stops at whatever the memory limit is, but that the rest of the program's execution still obeys the total

heap size rule set by GOGC.

This observation leads to another interesting detail: even when GOGC is set to off, the memory limit is still respected! In fact, this particular configuration represents a *maximization of resource economy* because it sets the minimum GC frequency required to maintain some memory limit. In this case, *all* of the program's execution has the heap size rise to meet the memory limit.

Now, while the memory limit is clearly a powerful tool, **the use of a memory limit does not come without a cost**, and certainly doesn't invalidate the utility of GOGC.

Consider what happens when the live heap grows large enough to bring total memory use close to the memory limit. In the steady state visualization above, try turning GOGC off and then slowly lowering the memory limit further and further to see what happens. Notice that the total time the application takes will start to grow in an unbounded manner as the GC is constantly executing to maintain an impossible memory limit.

This situation, where the program fails to make reasonable progress due to constant GC cycles, is called **thrashing**. It's particularly dangerous because it effectively stalls the program. Even worse, it can happen for exactly the same situation we were trying to avoid with GOGC: a large enough transient heap spike can cause a program to stall indefinitely! Try reducing the memory limit (around 30 MiB or lower) in the transient heap spike visualization and notice how the worst behavior specifically starts with the heap spike.

In many cases, an indefinite stall is worse than an out-of-memory condition, which tends to result in a much faster failure.

For this reason, the memory limit is defined to be **soft**. The Go runtime makes no guarantees that it will maintain this memory limit under all circumstances; it only promises some reasonable amount of effort. This relaxation of the memory limit is critical to avoiding thrashing behavior, because it gives the GC a way out: let memory use surpass the limit to avoid spending too much time in the GC.

How this works internally is the GC sets an upper limit on the amount of CPU time it can use over some time window (with some hysteresis for very short transient spikes in CPU use). This limit is currently set at roughly 50%, with a $2 * \texttt{GOMAXPROCS}$ CPU-second window. The consequence of limiting GC CPU time is that the GC's work is delayed, meanwhile the Go program may continue allocating new heap memory, even beyond the memory limit.

The intuition behind the 50% GC CPU limit is based on the worst-case impact on a program with ample available memory. In the case of a misconfiguration of the memory limit, where it is set too low mistakenly, the program will slow down at most by 2x, because the GC can't take more than 50% of its CPU time away.

> Note: the visualizations on this page do not simulate the GC CPU limit.

Suggested uses

While the memory limit is a powerful tool, and the Go runtime takes steps to mitigate the worst behaviors from misuse, it's still important to use it thoughtfully. Below is a collection of tidbits of advice about where the memory limit is most useful and applicable, and where it might cause more harm than good.

- **Do** take advantage of the memory limit when the execution environment of your Go program is entirely within your control, and the Go program is the only program with access to some set of resources (i.e. some kind of memory reservation, like a container memory limit).

  A good example is the deployment of a web service into containers with a fixed amount of available memory.

  In this case, a good rule of thumb is to leave an additional 5-10% of headroom to account for memory sources the Go runtime is unaware of.

- **Do** feel free to adjust the memory limit in real time to adapt to changing conditions.

  A good example is a cgo program where C libraries temporarily need to use substantially more memory.

- **Don't** set GOGC to off with a memory limit if the Go program might share some of its limited memory with other programs, and those programs are generally decoupled from the Go program. Instead, keep the memory limit since it may help to curb undesirable transient behavior, but set GOGC to some smaller, reasonable value for the average case.

  While it may be tempting to try and "reserve" memory for co-tenant programs, unless the programs are fully synchronized (e.g. the Go program calls some subprocess and blocks while its callee executes), the result will be less reliable as inevitably both programs will need more memory. Letting the Go program use less memory when it doesn't need it will generate a more reliable result overall. This advice also applies to overcommit situations, where the sum of memory limits of containers running on one machine may exceed the actual physical memory available to the machine.

- **Don't** use the memory limit when deploying to an execution environment you don't control, especially when your program's memory use is proportional to its inputs.

  A good example is a CLI tool or a desktop application. Baking a memory limit into the program when it's unclear what kind of inputs it might be fed, or how much memory might be available on the system can lead to confusing crashes and poor performance. Plus, an advanced end-user can always set a memory limit if they wish.

- **Don't** set a memory limit to avoid out-of-memory conditions when a program is already close to its environment's memory limits.

This effectively replaces an out-of-memory risk with a risk of severe application slowdown, which is often not a favorable trade, even with the efforts Go makes to mitigate thrashing. In such a case, it would be much more effective to either increase the environment's memory limits (and *then* potentially set a memory limit) or decrease GOGC (which provides a much cleaner trade-off than thrashing-mitigation does).

Latency

The visualizations in this document have modeled the application as paused while the GC is executing. GC implementations do exist that behave this way, and they're referred to as "stop-the-world" GCs.

The Go GC, however, is not fully stop-the-world and does most of its work concurrently with the application. This is primarily to reduce application *latencies*. Specifically, the end-to-end duration of a single unit of computation (e.g. a web request). Thus far, this document mainly considered application *throughput* (e.g. web requests handled per second). Note that each example in the GC cycle section focused on the total CPU duration of an executing program. However, such a duration is far less meaningful for say, a web service. While throughput is still important for a web service (i.e. queries per second), often the latency of each individual request matters even more.

In terms of latency, a stop-the-world GC may require a considerable length of time to execute both its mark and sweep phases, during which the application, and in the context of a web service, any in-flight request, is unable to make further progress. Instead, the Go GC avoids making the length of any global application pauses proportional to the size of the heap, and that the core tracing algorithm is performed while the application is actively executing. (The pauses are more strongly proportional to GOMAXPROCS algorithmically, but most commonly are dominated by the time it takes to stop running goroutines.) Collecting concurrently is not without cost: in practice it often leads to a design with lower throughput than an equivalent stop-the-world garbage collector. However, it's important to note that *lower latency does not inherently mean lower throughput*, and the performance of the Go garbage collector has steadily improved over time, in both latency and throughput.

The concurrent nature of Go's current GC does not invalidate anything discussed in this document so far: none of the statements relied on this design choice. GC frequency is still the primary way the GC trades off between CPU time and memory for throughput, and in fact, it also takes on this role for latency. This is because most of the costs for the GC are incurred while the mark phase is active.

The key takeaway then, is that **reducing GC frequency may also lead to latency improvements**. This applies not only to reductions in GC frequency from modifying tuning parameters, like increasing GOGC and/or the memory limit, but also applies to the optimizations described in the optimization guide.

However, latency is often more complex to understand than throughput, because it is a product of the moment-to-moment execution of the program and not just an aggregation of costs. As a result, the connection between latency and GC frequency is less direct. Below is a list of possible sources of latency for those inclined to dig deeper.

1. Brief stop-the-world pauses when the GC transitions between the mark and sweep phases,
2. Scheduling delays because the GC takes 25% of CPU resources when in the mark phase,
3. User goroutines assisting the GC in response to a high allocation rate,
4. Pointer writes requiring additional work while the GC is in the mark phase, and
5. Running goroutines must be suspended for their roots to be scanned.

These latency sources are visible in execution traces, except for pointer writes requiring additional work.

Finalizers, cleanups, and weak pointers

Garbage collection provides the illusion of infinite memory using only finite memory. Memory is allocated but never explicitly freed, which enables simpler APIs and concurrent algorithms compared to bare-bones manual memory management. (Some languages with manually managed memory use alternative approaches such as "smart pointers" and compile-time ownership tracking to ensure that objects are freed, but these features are deeply embedded into the API design conventions in these languages.)

Only the live objects—those reachable from a global variable or a computation in some goroutine—can affect the behavior of the program. Any time after an object becomes unreachable ("dead"), it may be safely recycled by the GC. This allows for a wide variety of GC designs, such as the tracing design used by Go today. The death of an object is not an observable event at the language level.

However, Go's runtime library provides three features that break that illusion: cleanups, weak pointers, and finalizers. Each of these features provides some way to observe and react to object death, and in the case of finalizers, even reverse it. This of course complicates Go programs and adds an additional burden to the GC implementation. Nonetheless, these features exist because they are useful in a variety of circumstances, and Go programs use them and benefit from them all the time.

For the details of each feature, refer to its package documentation (runtime.AddCleanup, weak.Pointer, runtime.SetFinalizer). Below is some general advice for using these features, outlines of common issues you can run into with each feature, and advice for testing uses of these features.

General advice

- **Write unit tests.**

The exact timing of cleanups, weak pointers, and finalizers can be difficult to predict, and it's easy to convince yourself that everything works, even after many consecutive executions. But it's also easy to make subtle mistakes. Writing tests for them can be tricky, but given that they're so subtle to use, testing is even more important usual.

- **Avoid using these features directly in typical Go code.**

  These are low-level features with subtle restrictions and behaviors. For instance, there's no guarantee cleanups or finalizers will be run at program exit, or at all for that matter. The long comments in their API documentation should be seen as a warning. The vast majority of Go code does not benefit from using these features directly, only indirectly.

- **Encapsulate the use of these mechanisms within a package.**

  Where possible, do not allow the use of these mechanisms to leak into the public API of your package; provide interfaces that make it hard or impossible for users to misuse them. For example, instead of asking the user to set up a cleanup on some C-allocated memory to free it, write a wrapper package and hide that detail inside.

- **Restrict access to objects that have finalizers, cleanups, and weak pointers to the package that created and applied them.**

  This is related to the previous point, but is worth calling out explicitly, since it's a very powerful pattern for using these features in a less error-prone way. For example, the unique package uses weak pointers under the hood, but completely encasulates the objects that are weakly pointed-to. Those values can never be mutated by the rest of the application, it can only be copied through the Value method, preserving the illusion of infinite memory for package users.

- **Prefer cleaning up non-memory resources deterministically when possible, with finalizers and cleanups as a fallback.**

  Cleanups and finalizers are a good fit for memory resources such as memory allocated externally, like from C, or references to an `mmap` mapping. Memory allocated by C's malloc must eventually be freed by C's free. A finalizer that calls `free`, attached to a wrapper object for the C memory, is a reasonable way to ensure that C memory is eventually reclaimed as a consequence of garbage collection.

  However, non-memory resources, like file descriptors, tend to be subject to system limits that the Go runtime is generally unaware of. In addition, the timing of the garbage collector in a given Go program is usually something a package author has little control over (for instance, how often the GC runs is controlled by GOGC, which can be set by operators to a variety of different values in practice). These two facts conspire to make cleanups and finalizers a bad fit to use as the only mechanism for releasing non-memory resources.

If you're a package author exposing an API that wraps some non-memory resource, consider providing an explicit API for releasing the resource deterministically (through a `Close` method, or something similar), rather than relying on the garbage collector through cleanups or finalizers. Instead, prefer to use cleanups and finalizers as a best-effort handler for programmer mistakes, either by cleaning up the resource anyway like os.File does, or by reporting the failure to deterministically clean up back to the user.

- **Prefer cleanups to finalizers.**

  Historically, finalizers were added to simplify the interface between Go code and C code and to clean up non-memory resources. The intended use was to apply them to wrapper objects that owned C memory or some other non-memory resource, so that the resource could be released once Go code was done using it. These reasons at least partially explain why finalizers are narrowly scoped, why any given object can only have one finalizer, and why that finalizer must be attached to the first byte of the object only. This limitation already stifles some use-cases. For example, any package that wishes to internally cache some information about an object passed to it cannot clean up that information once the object is gone.

  But worse than that, finalizers are inefficient and error-prone due to the fact that they resurrect the object they're attached to, so that it can be passed to the finalizer function (and even continue to live beyond that, too). This simple fact means that if the object is part of a reference cycle it can never be freed, and the memory backing the object cannot be reused until at least until the following garbage collection cycle.

  Because finalizers resurrect objects, though, they do have a better-defined execution order than cleanups. For this reason, finalizers are still potentially (but rarely) useful for cleaning up structures that have complex destruction ordering requirements.

  But for all other uses in Go 1.24 and beyond, we recommend you use cleanups because they are more flexible, less error-prone, and more efficient than finalizers.

Common cleanup issues

- Objects with attached cleanups must not be reachable from the cleanup function (for example, through a captured local variable). This will prevent the object from being reclaimed and the cleanup from ever running.

```
f := new(myFile)
f.fd = syscall.Open(...)
runtime.AddCleanup(f, func(fd int) {
        syscall.Close(f.fd) // Mistake: We reference f, so this cleanup won't
}, f.fd)
```

- Objects with attached cleanups must not be reachable from the argument to the cleanup function. This will prevent the object from being reclaimed and the cleanup from ever

running.

```
f := new(myFile)
f.fd = syscall.Open(...)
runtime.AddCleanup(f, func(f *myFile) {
        syscall.Close(f.fd)
}, f) // Mistake: We reference f, so this cleanup wouldn't ever run. This spec
```

- Finalizers have a well-defined execution order, but cleanups do not. Cleanups can also run concurrently with one another.
- Long running cleanups should create a goroutine to avoid blocking the execution of other cleanups.
- `runtime.GC` will not wait until cleanups for unreachable objects are executed, only until they are all queued.

Common weak pointer issues

- Weak pointers can begin returning `nil` from their `Value` method at unexpected times. Always guard the call to `Value` with a `nil` check and have a backup plan.
- When weak pointers are used as map keys, they do not affect the reachability of map values. Therefore, if a weak pointer map key points to an object that is also reachable from the map value, that object will still be considered reachable.

Common finalizer issues

- Objects with attached finalizers must not be reachable from themselves by any path (in other words, they cannot be in a reference cycle). This will prevent the object from being reclaimed and the finalizer from ever running.

```
f := new(myCycle)
f.self = f // Mistake: f is reachable from f, so this finalizer would never ru
runtime.SetFinalizer(f, func(f *myCycle) {
        ...
})
```

- Objects with attached finalizers must not be reachable from the finalizer function (for example, through a captured local variable). This will prevent the object from being reclaimed and the finalizer from ever running.

```
f := new(myFile)
f.fd = syscall.Open(...)
runtime.SetFinalizer(f, func(_ *myFile) {
        syscall.Close(f.fd) // Mistake: We reference the outer f, so this clea
})
```

- Reference chains of objects with attached finalizers (say, in a linked list) take, at minimum, as many GC cycles as there are objects in the chain to clean them all up. Keep finalizers shallow!

```
// Mistake: reclaiming this linked list will take at least 10 GC cycles.
node := new(linkedListNode)
for range 10 {
        tmp := new(linkedListNode)
        tmp.next = node
        node = tmp
        runtime.SetFinalizer(node, func(node *linkedListNode) {
                ...
        })
}
```

- Avoid placing finalizers on objects returned at package boundaries. This makes it possible for users of your package to call `runtime.SetFinalizer` to mutate the finalizer on the object you return, which can be an unexpected behavior that users of your package may end up relying on.

- Long running finalizers should create a new goroutine to avoid blocking the execution of other finalizers.

- `runtime.GC` will not wait until finalizers for unreachable objects are executed, only until they are all queued.

Testing object death

When using these features, it can sometimes be tricky to write tests for code that uses them. Here are some tips for writing robust tests for code that uses these features.

- Avoid running such tests in parallel with other tests. It helps a lot to increase determinism as much as possible and to have a good handle on the state of the world at any given time.
- Use `runtime.GC` to establish a baseline upon entering the test. Use `runtime.GC` to force weak pointers to `nil`, and to queue up cleanups and finalizers to run.

- `runtime.GC` does not wait for cleanups and finalizers to run, it only queues them.

  To write the most robust tests possible, inject a way to block on a cleanup or finalizer from your test (for example, pass an optional channel to the cleanup and/or finalizer from the test, and write to the channel once it has finished executing). If this is too hard or impossible, an alternative is to spin on a particular post-cleanup state to be true. For example, the `os` tests call `runtime.Gosched` in a loop that checks whether a file has been closed, once it becomes unreachable.

- If writing tests for using finalizers, and you have a chain of objects that use finalizers, you will need at minimum the length of the deepest chain the test can create of `runtime.GC`

calls to ensure all the finalizers run.

- Test in race mode to discover races between concurrent cleanups, and between cleanup and finalizer code and the rest of the codebase.

Additional resources

While the information presented above is accurate, it lacks the detail to fully understand costs and trade-offs in the Go GC's design. For more information, see the following additional resources.

- The GC Handbook—An excellent general resource and reference on garbage collector design.
- TCMalloc—Design document for the C/C++ memory allocator TCMalloc, which the Go memory allocator is based on.
- Go 1.5 GC announcement—The blog post announcing the Go 1.5 concurrent GC, which describes the algorithm in more detail.
- Getting to Go—An in-depth presentation about the evolution of Go's GC design up to 2018.
- Go 1.5 concurrent GC pacing—Design document for determining when to start a concurrent mark phase.
- Smarter scavenging—Design document for revising the way the Go runtime returns memory to the operating system.
- Scalable page allocator—Design document for revising the way the Go runtime manages memory it gets from the operating system.
- GC pacer redesign (Go 1.18)—Design document for revising the algorithm to determine when to start a concurrent mark phase.
- Soft memory limit (Go 1.19)—Design document for the soft memory limit.

A note about virtual memory

This guide has largely focused on the physical memory use of the GC, but a question that comes up regularly is what exactly that means and how it compares to virtual memory (typically presented in programs like `top` as "VSS").

Physical memory is memory housed in the actual physical RAM chip in most computers. Virtual memory is an abstraction over physical memory provided by the operating system to isolate programs from one another. It's also typically acceptable for programs to reserve virtual address space that doesn't map to any physical addresses at all.

Because virtual memory is just a mapping maintained by the operating system, it is typically very cheap to make large virtual memory reservations that don't map to physical memory.

The Go runtime generally relies upon this view of the cost of virtual memory in a few ways:

- The Go runtime never deletes virtual memory that it maps. Instead, it uses special operations that most operating systems provide to explicitly release any physical memory resources associated with some virtual memory range.

  This technique is used explicitly to manage the memory limit and return memory to the operating system that the Go runtime no longer needs. The Go runtime also releases memory it no longer needs continuously in the background. See the additional resources for more information.

- On 32-bit platforms, the Go runtime reserves between 128 MiB and 512 MiB of address space up-front for the heap to limit fragmentation issues.

- The Go runtime uses large virtual memory address space reservations in the implementation of several internal data structures. On 64-bit platforms, these typically have a minimum virtual memory footprint of about 700 MiB. On 32-bit platforms, their footprint is negligible.

As a result, virtual memory metrics such as "VSS" in `top` are typically not very useful in understanding a Go program's memory footprint. Instead, focus on "RSS" and similar measurements, which more directly reflect physical memory usage.

Optimization guide

Identifying costs

Before trying to optimize how your Go application interacts with the GC, it's important to first identify that the GC is a major cost in the first place.

The Go ecosystem provides a number of tools for identifying costs and optimizing Go applications. For a brief overview of these tools, see the guide on diagnostics. Here, we'll focus on a subset of these tools and a reasonable order to apply them in in order to understand GC impact and behavior.

1. **CPU profiles**

   A good place to start is with CPU profiling. CPU profiling provides an overview of where CPU time is spent, though to the untrained eye it may be difficult to identify the magnitude of the role the GC plays in a particular application. Luckily, understanding how the GC fits in mostly boils down to knowing what different functions in the `runtime` package mean. Below is a useful subset of these functions for interpreting CPU profiles.

   Note that the functions listed below are not leaf functions, so they may not show up in the default the `pprof` tool provides with the `top` command. Instead, use the `top -cum` command or use the `list` command on these functions directly and focus on the cumulative percent column.

- **`runtime.gcBgMarkWorker`**: Entrypoint to the background mark worker goroutines. Time spent here scales with GC frequency and the complexity and size of the object graph. It represents a baseline for how much time the application spends marking and scanning.

  Note that within these goroutines, you will find calls to `runtime.gcDrainMarkWorkerDedicated`, `runtime.gcDrainMarkWorkerFractional`, and `runtime.gcDrainMarkWorkerIdle`, which indicate worker type. In a largely idle Go application, the Go GC is going to use up additional (idle) CPU resources to get its job done faster, which is indicated with the `runtime.gcDrainMarkWorkerIdle` symbol. As a result, time here may represent a large fraction of CPU samples, which the Go GC believes are free. If the application becomes more active, CPU time in idle workers will drop. One common reason this can happen is if an application runs entirely in one goroutine but `GOMAXPROCS` is >1.

- **`runtime.mallocgc`**: Entrypoint to the memory allocator for heap memory. A large amount of cumulative time spent here (>15%) typically indicates a lot of memory being allocated.

- **`runtime.gcAssistAlloc`**: Function goroutines enter to yield some of their time to assist the GC with scanning and marking. A large amount of cumulative time spent here (>5%) indicates that the application is likely out-pacing the GC with respect to how fast it's allocating. It indicates a particularly high degree of impact from the GC, and also represents time the application spend marking and scanning. Note that this is included in the `runtime.mallocgc` call tree, so it will inflate that as well.

2. **Execution traces**

   While CPU profiles are great for identifying where time is spent in aggregate, they're less useful for indicating performance costs that are more subtle, rare, or related to latency specifically. Execution traces on the other hand provide a rich and deep view into a short window of a Go program's execution. They contain a variety of events related to the Go GC and specific execution paths can be directly observed, along with how the application might interact with the Go GC. All the GC events tracked are conveniently labeled as such in the trace viewer.

   See the documentation for the `runtime/trace` package for how to get started with execution traces.

3. **GC traces**

   When all else fails, the Go GC provides a few different specific traces that provide much deeper insights into GC behavior. These traces are always printed directly to STDERR, one line per GC cycle, and are configured through the `GODEBUG` environment variable that all

Go programs recognize. They're mostly useful for debugging the Go GC itself since they require some familiarity with the specifics of the GC's implementation, but nonetheless can occasionally be useful to gain a better understanding of GC behavior.

The core GC trace is enabled by setting `GODEBUG=gctrace=1`. The output produced by this trace is documented in the environment variables section in the documentation for the `runtime` package.

A supplementary GC trace called the "pacer trace" provides even deeper insights and is enabled by setting `GODEBUG=gcpacertrace=1`. Interpreting this output requires an understanding of the GC's "pacer" (see additional resources), which is outside the scope of this guide.

Eliminating heap allocations

One way to reduce costs from the GC is to have the GC manage fewer values to begin with. The techniques described below can produce some of the largest improvements in performance, because as the GOGC section demonstrated, the allocation rate of a Go program is a major factor in GC frequency, the key cost metric used by this guide.

Heap profiling

After identifying that the GC is a source of significant costs, the next step in eliminating heap allocations is to find out where most of them are coming from. For this purpose, memory profiles (really, heap memory profiles) are very useful. Check out the documentation for how to get started with them.

Memory profiles describe where in the program heap allocations come from, identifying them by the stack trace at the point they were allocated. Each memory profile can break down memory in four ways.

- `inuse_objects`—Breaks down the number of objects that are live.
- `inuse_space`—Breaks down live objects by how much memory they use in bytes.
- `alloc_objects`—Breaks down the number of objects that have been allocated since the Go program began executing.
- `alloc_space`—Breaks down the total amount of memory allocated since the Go program began executing.

Switching between these different views of heap memory may be done with either the `–sample_index` flag to the `pprof` tool, or via the `sample_index` option when the tool is used interactively.

Note: memory profiles by default only sample a subset of heap objects so they will not contain information about every single heap allocation. However, this is sufficient to find hot-spots. To change the sampling rate, see `runtime.MemProfileRate`.

For the purposes of reducing GC costs, `alloc_space` is typically the most useful view as it directly corresponds to the allocation rate. This view will indicate allocation hot spots that would provide the most benefit.

Escape analysis

Once candidate heap allocation sites have been identified with the help of heap profiles, how can they be eliminated? The key is to leverage the Go compiler's escape analysis to have the Go compiler find alternative, and more efficient storage for this memory, for example in the goroutine stack. Luckily, the Go compiler has the ability to describe why it decides to escape a Go value to the heap. With that knowledge, it becomes a matter of reorganizing your source code to change the outcome of the analysis (which is often the hardest part, but outside the scope of this guide).

As for how to access the information from the Go compiler's escape analysis, the simplest way is through a debug flag supported by the Go compiler that describes all optimizations it applied or did not apply to some package in a text format. This includes whether or not values escape. Try the following command, where `[package]` is some Go package path.

```
$ go build -gcflags=-m=3 [package]
```

This information can also be visualized as an overlay in an LSP-capable editor; it is exposed as a code action. For example, in VS Code, invoke the "Source Action... > Show compiler optimization details" command to enable diagnostics for the current package. (You can also run the "Go: Toggle compiler optimization details" command.) Use this configuration setting to control which annotations are displayed:

1. Enable the overlay for escape analysis by setting `ui.diagnostic.annotations` to include `escape` .

Finally, the Go compiler provides this information in a machine-readable (JSON) format that may be used to build additional custom tooling. For more information on that, see the documentation in the source Go code.

Implementation-specific optimizations

The Go GC is sensitive to the demographics of live memory, because a complex graph of objects and pointers both limits parallelism and generates more work for the GC. As a result, the GC contains a few optimizations for specific common structures. The most directly useful ones for performance optimization are listed below.

> Note: Applying the optimizations below may reduce the readability of your code by obscuring intent, and may fail to hold up across Go releases. Prefer to apply these optimizations only in the places they matter most. Such places may be identified by using the tools listed in the section on identifying costs.

- Pointer-free values are segregated from other values.

  As a result, it may be advantageous to eliminate pointers from data structures that do not strictly need them, as this reduces the cache pressure the GC exerts on the program. As a result, data structures that rely on indices over pointer values, while less well-typed, may perform better. This is only worth doing if it's clear that the object graph is complex and the GC is spending a lot of time marking and scanning.

- The GC will stop scanning values at the last pointer in the value.

  As a result, it may be advantageous to group pointer fields in struct-typed values at the beginning of the value. This is only worth doing if it's clear the application spends a lot of its time marking and scanning. (In theory the compiler can do this automatically, but it is not yet implemented, and struct fields are arranged as written in the source code.)

Furthermore, the GC must interact with nearly every pointer it sees, so using indices into an slice, for example, instead of pointers, can aid in reducing GC costs.

Linux transparent huge pages (THP)

When a program accesses memory, the CPU needs to translate the virtual memory addresses it uses into physical memory addresses that refer to the data it was trying to access. To do this, the CPU consults the "page table," a data structure that represents the mapping from virtual to physical memory, managed by the operating system. Each entry in the page table represents an indivisible block of physical memory called a page, hence the name.

Transparent huge pages (THP) is a Linux feature that transparently replaces pages of physical memory backing contiguous virtual memory regions with bigger blocks of memory called huge pages. By using bigger blocks, fewer page table entries are needed to represent the same memory region, improving page table lookup times. However, bigger blocks mean more waste if only a small part of the huge page is used by the system.

When running Go programs in production, enabling transparent huge pages on Linux can improve throughput and latency at the cost of additional memory use. Applications with small heaps tend not to benefit from THP and may end up using a substantial amount of additional memory (as high as 50%). However, applications with big heaps (1 GiB or more) tend to benefit quite a bit (up to 10% throughput) without very much additional memory overhead (1-2% or less). Being aware of your THP settings in either case can be helpful, and experimentation is always recommended.

One can enable or disable transparent huge pages in a Linux environment by modifying `/sys/kernel/mm/transparent_hugepage/enabled`. See the official Linux admin guide for more details. If you choose to have your Linux production environment enable transparent huge pages, we recommend the following additional settings for Go programs.

- Set `/sys/kernel/mm/transparent_hugepage/defrag` to `defer` or `defer+madvise`.

  This setting controls how aggressively a Linux kernel coalesces regular pages into huge pages. `defer` tells the kernel to coalesce huge pages lazily and in the background. A more aggressive setting can induce stalls in memory constrained systems and can often hurt application latencies. `defer+madvise` is like `defer`, but is friendlier to other applications on the system that request huge pages explicitly and require them for performance.

- Set `/sys/kernel/mm/transparent_hugepage/khugepaged/max_ptes_none` to `0`.

  This setting controls how many additional pages the Linux kernel daemon can allocate when trying to allocate a huge page. The default setting is maximally aggressive, and can often undo work the Go runtime does to return memory to the OS. Before Go 1.21, the Go runtime tried to mitigate the negative effects of the default setting, but it came with a CPU cost. With Go 1.21+ and Linux 6.2+, the Go runtime no longer mutates huge page state.

  If you experience an increase in memory usage when upgrading to Go 1.21.1 or later, try applying this setting; it will likely resolve your issue. As an additional workaround, you can call the `Prctl` function with `PR_SET_THP_DISABLE` to disable huge pages at the process level, or you can set `GODEBUG=disablethp=1` (to be added in Go 1.21.6 and Go 1.22) to disable huge pages for heap memory. Note that the `GODEBUG` setting may be removed in a future release.

Appendix

Additional notes on GOGC

The GOGC section claimed that doubling GOGC doubles heap memory overheads and halves GC CPU costs. To see why, let's break it down mathematically.

Firstly, the heap target sets a target for the total heap size. This target, however, mainly influences the new heap memory, because the live heap is fundamental to the application.

$$\textit{Target heap memory} = \textit{Live heap} + (\textit{Live heap} + \textit{GC roots}) * \textit{GOGC} / 100$$

$$\textit{Total heap memory} = \textit{Live heap} + \textit{New heap memory}$$

$$\Rightarrow$$

$$\textit{New heap memory} = (\textit{Live heap} + \textit{GC roots}) * \textit{GOGC} / 100$$

From this we can see that doubling GOGC would also double the amount of new heap memory that application will allocate each cycle, which captures heap memory overheads. Note that *Live*

heap + GC roots is an approximation of the amount of memory the GC needs to scan.

Next, let's look at GC CPU cost. Total cost can be broken down as the cost per cycle, times GC frequency over some time period T.

*Total GC CPU cost = (GC CPU cost per cycle) * (GC frequency) * T*

GC CPU cost per cycle can be derived from the GC model:

*GC CPU cost per cycle = (Live heap + GC roots) * (Cost per byte) + Fixed cost*

Note that sweep phase costs are ignored here as mark and scan costs dominate.

The steady state is defined by a constant allocation rate and a constant cost per byte, so in the steady state we can derive a GC frequency from this new heap memory:

*GC frequency = (Allocation rate) / (New heap memory) = (Allocation rate) / ((Live heap + GC roots) * GOGC / 100)*

Putting this together, we get the full equation for the total cost:

*Total GC CPU cost = (Allocation rate) / ((Live heap + GC roots) * GOGC / 100) * ((Live heap + GC roots) * (Cost per byte) + Fixed cost) * T*

For a sufficiently large heap (which represents most cases), the marginal costs of a GC cycle dominate the fixed costs. This allows for a significant simplification of the total GC CPU cost formula.

*Total GC CPU cost = (Allocation rate) / (GOGC / 100) * (Cost per byte) * T*

From this simplified formula, we can see that if we double GOGC, we halve total GC CPU cost. (Note that the visualizations in this guide do simulate fixed costs, so the GC CPU overheads reported by them will not exactly halve when GOGC doubles.) Furthermore, GC CPU costs are largely determined by allocation rate and the cost per byte to scan memory. For more information on how to reduce these costs specifically, see the optimization guide.

> Note: there exists a discrepancy between the size of the live heap, and the amount of that memory the GC actually needs to scan: the same size live heap but with a different structure will result in a different CPU cost, but the same memory cost, resulting a different trade-off. This is why the structure of the heap is part of the definition of the steady state. The heap target should arguably only include the scannable live heap as a closer approximation of memory the GC needs to scan, but this leads to degenerate behavior when there's a very small amount of scannable live heap but the live heap is otherwise large.